DS8001 Project Proposal: Permutation-Based Approximate kNN Search for Mixed-Type Hospital Encounters

Jared Smouter and Amarnadh Nalla | Project path: **Empirical Analysis** | Group of two

1. Problem and questions. Exact k-nearest-neighbor (kNN) search computes a distance to all N indexed encounters. We study, as a purely algorithmic question (no clinical claim), how permutation-based approximate indexes trade recall against distance computations, index size and build cost under a mixed-type distance. The dataset is widely used for readmission prediction; our contribution is an algorithmic study of approximate similarity search on it. **RQ1.** How do the number of pivots n, the retained permutation length L and the re-ranking budget c affect recall@10 and query cost, and how do these costs scale with N? **RQ2.** At matched L and c, when does a truncated MI-File beat a truncated permutation scan, and how does random versus farthest-first pivot selection change this?

2. Methods. Implemented by us: (i) **brute-force exact kNN** (ground truth, ties broken by encounter id); (ii) a **truncated permutation scan** (TPS): each object stores the ranks of its L nearest pivots (rank L+1 otherwise); the query scores all objects by the Spearman footrule over the n pivots, keeps the best c and re-ranks them with the true distance (exact only within the candidate set); (iii) a **truncated MI-File** (Amato and Savino, 2008): per-pivot posting lists of (object, rank), with the same score and re-ranking, so only objects sharing a retained pivot are touched; (iv) random versus farthest-first pivot selection. Library baseline (not counted as ours): scikit-learn **BallTree** with the Manhattan metric on an exact weighted-L1 encoding of the distance, validated against brute force. Complexity analysis: build O(Nn) distances plus ranking, storage O(NL), query n pivot distances plus candidate scoring plus c re-ranking distances. **Distance:** six equal-weight feature groups (age, utilization, admission/discharge, ICD-9 diagnosis chapters, labs, medications); numeric features use scale-normalized absolute difference, nominal ones 0/1 mismatch, missing values get a fixed state or median plus indicator (no pairwise omission). With fixed scales and weights this is a metric, which we verify (identity, triangle inequality) on sampled triples.

3. Data. Diabetes 130-US Hospitals (1999–2008) (Strack et al., 2014; UCI, CC BY 4.0): 101,766 encounters, 71,518 patients. We exclude IDs, `readmitted`, race, gender and `weight`. Patients are split 90/5/5 into index/validation/test with a fixed seed, so no patient appears in both index and queries; scales and medians are fit on the index partition only.

4. Experiments and metrics. Nested index sizes N = 10k, 50k and the full index partition; 300 validation and 300 test queries (one encounter per patient). Grid: $n \in \{32, 64\}$, $L \in \{8, 16, 32\}$ ($L \le n$), $c/N \in \{0.5\%, 1\%, 5\%\}$, random and farthest-first pivots, 3 seeds each. We choose Pareto-optimal settings on validation, freeze them, and evaluate on test once. Metrics: recall@10 against brute force; distance computations, entries scored and postings read per query (machine-independent); serialized index size; build distances; median latency and peak memory (single-threaded, same machine). We report patient-level bootstrap 95% CIs, paired per-query differences, and seed variability separately from query variability. Deliverables: a Python 3.11+ package with README and fixed seeds, and a report relating the trade-offs to theory. Work split: J. Smouter – preprocessing, metric, brute force, BallTree; A. Nalla – TPS, MI-File, pivots; experiments, statistics and report are joint.

5. Implementation, validation and plan. All code is Python 3.11+ (NumPy, pandas, scikit-learn only for the BallTree baseline and pairwise L1 kernels), organized as a package (`data`, `brute`, pivots, TPS, MI-File, experiment runner) with fixed seeds, a README and pytest tests. Distances are computed in vectorized chunks; brute-force top-50 lists are cached per N. Correctness checks: metric axioms on sampled triples; BallTree results equal brute force; TPS and MI-File return exactly the brute-force answer when c = N and L = n; no patient in more than one partition. Primary coding is already under way: the loader, patient-grouped split, weighted-L1 encoder, brute-force oracle and metric tests are implemented. Plan: pivot permutations and TPS (mid-Nov), MI-File and BallTree validation (late Nov), full experiments, analysis and report (until Dec 7). We expect recall to rise with L and c and the MI-File to read far fewer entries than the scan at small L, at the risk of zero-overlap misses; we will test, not assume, this. **Threats to validity:** one dataset and one distance design, and recall is measured against the exact answer under our metric, not clinical similarity.

References

Chávez, E., Figueroa, K., and Navarro, G. (2008). Effective proximity retrieval by ordering permutations. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 30(9), 1647–1658.

Amato, G., and Savino, P. (2008). Approximate similarity search in metric spaces using inverted files. In *Proc. 3rd International Conference on Scalable Information Systems (InfoScale)*.

Mohamed, H., and Marchand-Maillet, S. (2015). Scalable indexing for big data processing. In *Big Data: Algorithms, Analytics, and Applications*. Chapman and Hall/CRC.

Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., and Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014, Article 781670. Data: UCI Machine Learning Repository, doi:10.24432/C5230J.